Assignment 1: Vision Transformers

Implementation and Analysis Report

CS 676 – Deep Learning
Spring 2026

Assigned: March 25, 2026 Due: April 23, 2026 at 11:59 PM

Contents

1 Overview **3**

2 Background: Vision Transformers **3**
2.1 From NLP to Vision . . . 3
2.2 The ViT Pipeline . . . 3
2.3 Why Study ViT? . . . 4

3 Code Template: `vit_template.py` **5**
3.1 File Structure . . . 5
3.2 Step-by-Step Usage . . . 6
3.3 Stub Conventions . . . 6

4 Tasks **7**
4.1 Part 1: Implementation (35 points) . . . 7
4.2 Part 2: Written Report (40 points) . . . 8
4.3 Part 3: Quantitative Analysis (25 points) . . . 9

5 JSON Log Format **10**

6 Checkpoint Format **10**

7 Using Git for This Assignment **11**
7.1 Initial Setup (Do This on Day 1) . . . 11
7.2 Recommended Commit Workflow . . . 11
7.3 Useful Git Commands . . . 12
7.4 Including `.git` in Your Zip . . . 13
7.5 Verifying Your Git History . . . 13

8 Submission Format **13**

9 Grading Summary **14**

10 Academic Integrity **14**

11 Tips and Common Pitfalls **15**

1 Overview

In this assignment you will implement a **Vision Transformer (ViT)** from scratch in PyTorch and train it on a subset of CIFAR-10. Once your model is working, a single command runs all ablation experiments and produces quantitative analysis outputs automatically. You will then write a **report** that interprets these results.

This assignment has three components:

1. **Implementation (35 pts)** – Implement the ViT architecture and training pipeline. Evaluated automatically.
2. **Written Report (40 pts)** – A PDF report analyzing the ablation experiments (Section A) and interpreting the quantitative analysis results (Section B). Graded manually by the teaching assistants.
3. **Quantitative Analysis (25 pts)** – Implement four analysis functions that compute interpretability metrics from your trained model. Evaluated automatically.

Everything runs on **CPU**. The model is intentionally small and the dataset subset is limited, so each training can run on a modern laptop.

Important

Your submission must contain a `.git` directory with at least **5 commits** spread across at least **2 different calendar days**. A dedicated script (`check_git.py`) will verify this automatically. Failing this check results in a **15-point deduction**. See Section 7 for detailed instructions.
Your submission must also include **report.pdf**. Missing the report means forfeiting the 40 report points.

2 Background: Vision Transformers

2.1 From NLP to Vision

The Transformer architecture [1] was originally introduced for sequence-to-sequence tasks in natural language processing. Its key innovation is the *self-attention* mechanism, which allows each element in a sequence to attend to every other element, capturing long-range dependencies without the sequential bottleneck of recurrent networks.

Dosovitskiy et al. (2021) asked a natural question: *can we apply a standard Transformer, with minimal modification, directly to images?* The resulting **Vision Transformer (ViT)** [2] showed that when pre-trained on sufficient data, a pure Transformer can match or outperform convolutional networks on image classification.

2.2 The ViT Pipeline

An image of shape $H \times W \times C$ is processed through the following stages:

1. Patch Extraction and Embedding. The image is divided into a grid of non-overlapping patches, each of size $P \times P$ pixels. With a 32×32 image and patch size $P = 4$, this gives

$N = (32/4)^2 = 64$ patches. Each patch (shape $P \times P \times C$) is flattened and linearly projected to a D-dimensional embedding vector using a learned projection. In practice this projection is implemented as a 2D convolution with kernel size P and stride P.

2. CLS Token and Positional Embeddings. Following BERT [3], a special learnable `[CLS]` token is prepended to the sequence of patch embeddings. The model is then asked to produce the classification output from this token's final representation.

Since Transformers are permutation-invariant (they have no built-in notion of order), *positional embeddings* are added to each token to encode spatial position. ViT uses simple **1-D learnable** positional embeddings over the sequence of $N + 1$ tokens (including CLS). Surprisingly, these 1-D embeddings learn to encode 2-D spatial structure: nearby patches tend to have more similar positional embeddings than distant ones (you will verify this in Part 3).

3. Transformer Encoder. The token sequence passes through L identical Transformer blocks. Each block is a **pre-norm** block:

$$\mathbf{x}' = \mathbf{x} + \mathrm{MHSA}(\mathrm{LN}(\mathbf{x})) \tag{1}$$

$$\mathbf{x}'' = \mathbf{x}' + \mathrm{MLP}(\mathrm{LN}(\mathbf{x}')) \tag{2}$$

where LN is Layer Normalization, MHSA is Multi-Head Self-Attention, and MLP is a two-layer feed-forward network with GELU activation. The residual connections ("$+\mathbf{x}$") stabilize training in deep networks.

Multi-Head Self-Attention (MHSA). Given a sequence of tokens $\mathbf{X} \in \mathbb{R}^{(N+1)\times D}$, MHSA projects each token into h heads of dimension $d_k = D/h$:

$$\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{3}$$

Each head independently computes attention weights and attends to values. The outputs of all h heads are concatenated and projected back to D dimensions. The $\sqrt{d_k}$ scaling prevents the dot products from growing large in magnitude and pushing the softmax into saturation.

4. Classification Head. After the final Transformer block and a last Layer Normalization, the CLS token's representation $\mathbf{z}_{\texttt{[CLS]}} \in \mathbb{R}^D$ is fed into a linear layer to produce class logits.

2.3 Why Study ViT?

Modern vision backbones—DeiT, Swin Transformer, ViT-22B, SAM, DINO, CLIP— are all built on the ViT foundation. Understanding how attention entropy varies across layers, how positional embeddings encode spatial structure, and how patch size trades off between resolution and compute are fundamental concepts that carry directly to research and production systems.

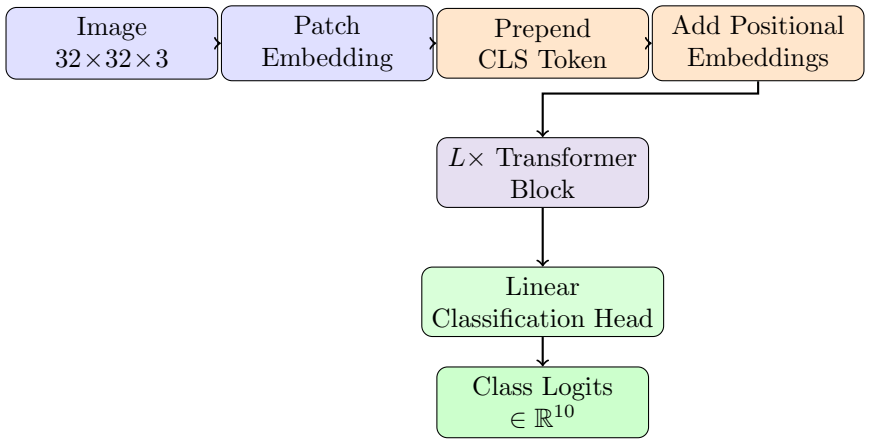

Figure 1: High-level pipeline of the Vision Transformer implemented in this assignment. The spatial 32×32 image is tokenized into patches, a CLS token is added, and the sequence passes through L Transformer blocks before a linear head produces class logits.

3 Code Template: `vit_template.py`

All your work lives in a single file: `vit_template.py`. Do not rename it. Do not modify function or class signatures. The autograder imports the file and calls specific functions by name.

3.1 File Structure

The template is organized into the following sections:

```
vit_template.py
  |
  |-- STUDENT_ID = 0          # <-- SET THIS FIRST
  |
  |-- get_seed()              # derives your personal seed from STUDENT_ID
  |-- set_all_seeds(seed)     # seeds Python / NumPy / PyTorch
  |
  |-- PatchEmbedding          # TODO 1.1  (you implement)
  |-- MultiHeadSelfAttention  # TODO 1.2  (you implement)
  |-- TransformerBlock        # TODO 1.3  (you implement)
  |-- VisionTransformer       # TODO 1.4  (you implement)
  |
  |-- get_cifar10_subset()    # TODO 1.5  (you implement)
  |-- train_model()           # TODO 1.6  (you implement)
  |
  |-- run_ablations()               # provided -- runs automatically
  |-- compute_attention_entropy()   # TODO 3.1  (you implement)
  |-- compute_pos_embed_correlation()# TODO 3.2  (you implement)
  |-- compute_per_class_accuracy()  # TODO 3.3  (you implement)
  |-- compute_attention_distance()  # TODO 3.4  (you implement)
  |
  |-- if __name__ == "__main__":
  |     # trains baseline then runs all ablations + analysis
```

3.2 Step-by-Step Usage

Step 0: Set your student ID. Open `vit_template.py` and find the line near the top:

```
STUDENT_ID = 0    # TODO: replace with your numeric student ID
```

Replace `0` with your actual numeric student ID (the same number that appears on your university ID card). Everything else—data subset selection, weight initialization, training trajectory—derives from this value. The autograder re-derives your seed from your submitted ID and re-computes your analysis quantities from your checkpoint. If the numbers do not match, you lose points.

Step 1: Install dependencies.

```
pip install torch>=2.0 torchvision>=0.15 numpy
```

No GPU is needed. No other packages are required.

Step 2: Run everything. After implementing the Part 1 stubs (Tasks 1.1–1.6), run:

```
python vit_template.py --mode all
```

This trains the baseline model, runs all ablation experiments, and computes all four analysis outputs automatically. Checkpoints are saved in `checkpoints/`, training logs in `training_log.json`, ablation logs in `ablation_logs/`, and analysis results in `analysis/`.

Step 3: Write your report. Inspect the generated JSON files and write a PDF report (`report.pdf`) as described in Section 4.2.

Validating JSON output. Before submitting, check that all JSON files are valid:

```
for f in training_log.json ablation_logs/*.json analysis/*.json; do
    python -m json.tool "$f" > /dev/null && echo "OK: $f" || echo "FAIL:
        $f"
done
```

Note

Baseline configuration: patch size 4, embedding dim 64, 2 attention heads, 4 Transformer layers, MLP hidden dim 128, dropout 0.0, batch size 64, learning rate 1×10^{-3}, 20 epochs. This produces roughly 200 000 parameters and trains in about 2–3 minutes on a laptop CPU.

3.3 Stub Conventions

Each stub contains a docstring in this format:

```
class PatchEmbedding(nn.Module):
    """
    Convert a batch of images into patch embeddings.

    Args:
        img_size  (int): spatial size of the input image (assumed square).
        patch_size(int): size of each patch (assumed square).
        in_chans  (int): number of input channels.
```

```
        embed_dim (int): output embedding dimension D.

    Forward:
        x: Tensor of shape (B, C, H, W)
    Returns:
        Tensor of shape (B, N, D)
        where N = (img_size // patch_size) ** 2
    """
    def __init__(self, img_size, patch_size, in_chans, embed_dim):
        super().__init__()
        # TODO: implement
        raise NotImplementedError

    def forward(self, x):
        # TODO: implement
        raise NotImplementedError
```

Replace each `raise NotImplementedError` with your implementation. Do *not* change the method signatures, class name, or docstring.

4 Tasks

4.1 Part 1: Implementation (35 points)

Implement the following components inside `vit_template.py`.

1.1 PatchEmbedding (5 pts)
Convert an image of shape $(B, 3, 32, 32)$ into patch embeddings of shape (B, N, D) using a convolutional projection with kernel size and stride both equal to the patch size. The convolution should output D channels, giving you an output of shape (B, D, G, G) where $G = 32/P$. Reshape and transpose to (B, N, D).

1.2 MultiHeadSelfAttention (10 pts)
Implement scaled dot-product attention with h heads from scratch. Your `forward` method must return a tuple `(output, attn_weights)` where `attn_weights` has shape $(B, h, N{+}1, N{+}1)$—the post-softmax attention matrix for all heads. **Do not use** `nn.MultiheadAttention`.

1.3 TransformerBlock (5 pts)
Implement the pre-norm residual block. The order of operations within each sub-block is: LayerNorm → MHSA (or MLP) → add residual. The MLP consists of two linear layers with GELU activation; apply dropout between them and after the second linear layer.

1.4 VisionTransformer (5 pts)
Assemble the full model: PatchEmbedding, a learnable CLS token (shape $1 \times 1 \times D$), learnable 1-D positional embeddings (shape $1 \times (N{+}1) \times D$), L TransformerBlocks, a final LayerNorm, and a linear classification head on the CLS token. The `forward` method must return `(logits, attn_list)` where `attn_list` is a Python list of L attention weight tensors, one per layer.

1.5 get_cifar10_subset (3 pts)
Load CIFAR-10. Using your personal seed, select a class-balanced *training* subset of 5 000 images (exactly 500 per class) by random sampling without replacement. The test set is the full 10 000-image test split (do not subsample it).

1.6 train_model (7 pts)
Implement the full training loop. Requirements: cross-entropy loss, AdamW optimizer, cosine annealing LR scheduler (`CosineAnnealingLR` with `T_max = epochs`). Log training loss and validation accuracy after each epoch. Save checkpoints at the caller-specified epoch indices. Write the JSON training log (format in Section 5).

4.2 Part 2: Written Report (40 points)

After running `python vit_template.py --mode all`, you will have `ablation_logs/` containing 7 training logs and `analysis/` containing 4 JSON result files.

Write a PDF report (`report.pdf`, max 6 pages) that addresses the following questions. Cite specific numbers from your JSON files to support every claim.

Section A: Ablation Analysis (25 pts)

A.1 Patch Size (10 pts)
Compare the final validation accuracy and total parameter count for patch sizes $\{2, 4, 8\}$ (from `patch_{2,4,8}.json`). Explain the trade-off between patch size, number of tokens N, and model expressiveness. Why does a smaller patch size not always produce higher accuracy?

A.2 Number of Layers (8 pts)
Compare final accuracy for depths $\{2, 4, 8\}$ (from `layers_{2,4,8}.json`). Does accuracy improve monotonically with depth on this small dataset? Discuss potential reasons for diminishing returns or over-fitting.

A.3 Positional Encoding (7 pts)
Compare the training curve and final accuracy of the model without positional embeddings (`no_pos_enc.json`) against the baseline. Explain *why* removing positional information affects performance, given that ViT is permutation-invariant without it.

Section B: Quantitative Analysis Interpretation (15 pts)

B.1 Attention Entropy (4 pts)
Report the per-layer entropy values from `analysis/attention_entropy.json`. Does entropy increase or decrease with depth? What does this tell you about how attention becomes more or less focused as information propagates through the layers?

B.2 Positional Embedding Correlation (4 pts)
Report the Pearson r from `analysis/pos_embed_correlation.json`. Is the correlation positive or negative, and why? What does the sign and magnitude tell you about whether the model has learned spatially coherent positional representations?

B.3 Per-Class Accuracy and Confusions (3 pts)
Report the per-class accuracies and the top-3 confusion pairs from `analysis/per_class_accuracy.json`. Which CIFAR-10 classes are hardest for the model and why might those specific pairs be confused?

B.4 Mean Attention Distance (4 pts)
Report the per-layer mean attention distances from `analysis/attention_distance.json`. Does the effective receptive field grow or shrink with depth? How does this compare to the behavior observed in convolutional networks?

4.3 Part 3: Quantitative Analysis (25 points)

Each analysis function loads your saved baseline checkpoint, computes the specified quantity over the *full test set*, and saves a JSON result to `analysis/`. The autograder re-derives the same quantities from your checkpoint and compares them within a tolerance of ± 0.02. Your report (Section B) must then interpret these outputs.

3.1 Attention Entropy (6 pts) – `compute_attention_entropy`:
For each of the 4 Transformer layers, compute the Shannon entropy of the CLS token's attention distribution, averaged over all heads and all test images:

$$H(p) = -\sum_i p_i \log_2 p_i, \qquad p_i \in [\epsilon, 1],\ \epsilon = 10^{-9}$$

Save `analysis/attention_entropy.json`:

```
{"layer_0": 3.21, "layer_1": 2.85, "layer_2": 2.61, "layer_3": 2.44}
```

3.2 Positional Embedding Distance Correlation (6 pts) – `compute_pos_embed_correlation`:
Extract the learned positional embeddings for the N patch positions (exclude CLS). Compute the pairwise cosine similarity matrix $S \in \mathbb{R}^{N\times N}$ and the pairwise Euclidean distance matrix $E \in \mathbb{R}^{N\times N}$ between 2-D grid coordinates (patch $k \mapsto (k//G,\ k \bmod G)$, $G = 32/P$). Report the Pearson correlation r between the strict upper triangles of S and E.

Save `analysis/pos_embed_correlation.json`:

```
{"pearson_r": -0.42, "num_pairs": 2016}
```

3.3 Per-Class Accuracy and Top-3 Confusions (5 pts) – `compute_per_class_accuracy`:
Evaluate the baseline on the full test set. Report per-class accuracy for all 10 CIFAR-10 classes (by numeric label 0–9). Identify the 3 most frequent misclassification pairs $(c_{\text{true}}, c_{\text{pred}})$.

Save `analysis/per_class_accuracy.json`:

```
{
  "class_accuracies": {"0": 0.55, "1": 0.62, ...},
  "top3_confusions": [[3, 5, 47], [5, 3, 41], [4, 7, 38]]
}
```

Each confusion entry is `[true_class, pred_class, count]`.

3.4 Mean Attention Distance (8 pts) – `compute_attention_distance`:
For each Transformer layer, extract the $N\times N$ patch-to-patch submatrix A' (rows and columns $1\ldots N$, excluding CLS). Row-normalize A' to sum to 1. For query patch i at grid position (r_i, c_i):

$$d_i = \sum_{j=1}^{N} A'_{ij} \cdot \sqrt{(r_i - r_j)^2 + (c_i - c_j)^2}$$

Average d_i over all N query patches, then over all heads, then over all test images.

Save `analysis/attention_distance.json`:

```
{"layer_0": 2.31, "layer_1": 2.55, "layer_2": 2.78, "layer_3": 3.01}
```

5 JSON Log Format

Every training log file (baseline and ablations) must use exactly this schema:

```
{
  "student_id": 123456789,
  "seed": 42,
  "config": {
    "patch_size": 4,
    "embed_dim": 64,
    "num_heads": 2,
    "num_layers": 4,
    "mlp_dim": 128,
    "dropout": 0.0,
    "lr": 0.001,
    "batch_size": 64,
    "epochs": 20
  },
  "history": [
    {
      "epoch": 1,
      "train_loss": 2.15,
      "val_accuracy": 0.18,
      "epoch_time_sec": 8.3
    },
    ...
  ],
  "final_val_accuracy": 0.45,
  "total_params": 202890
}
```

Note

Round all floating-point values to 4 decimal places before writing JSON. The autograder checks the JSON schema structure exactly.

6 Checkpoint Format

Save checkpoints using `torch.save` with a dictionary containing:

```
torch.save({
    "model_state_dict": model.state_dict(),
    "config": {
        "patch_size": ...,
        "embed_dim": ...,
        "num_heads": ...,
        "num_layers": ...,
        "mlp_dim": ...,
        "dropout": ...,
    },
    "epoch": 20,
    "student_id": STUDENT_ID,
}, "checkpoints/baseline_epoch_20.pt")
```

The `config` dict must match the config used during training exactly. The autograder reconstructs the model architecture from this dict before loading `model_state_dict`.

7 Using Git for This Assignment

Important

Your submission zip must include the `.git` directory. A script (`check_git.py`) will inspect your commit history and verify that you have at least **5 commits** spread over at least **2 calendar days**. Submitting a single commit with all code is a **15-point deduction** and signals a potential academic integrity concern.

Git is a version control system that tracks changes to your files over time. For this assignment, you are expected to use Git as you normally would when developing a software project: commit incrementally as you finish each piece, not all at once at the end.

7.1 Initial Setup (Do This on Day 1)

Navigate to your project directory and initialize a Git repository:

```
cd UBNumber/
git init
git add vit_template.py
git commit -m "initial commit: set STUDENT_ID and project structure"
```

Note

If you have never used Git before, you may need to set your name and email once on your machine:

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

7.2 Recommended Commit Workflow

Commit after each meaningful milestone. Here is a suggested progression *spread over multiple working sessions*:

Commit 1. *(Day 1)* Set `STUDENT_ID`, scaffold, and verify `get_seed()` runs without errors.

```
git add vit_template.py
git commit -m "set STUDENT_ID; verify seed generation"
```

Commit 2. *(Day 1 or 2)* Implement `PatchEmbedding` and `MultiHeadSelfAttention`. Verify shapes match docstrings with a quick test:

```
git add vit_template.py
git commit -m "implement PatchEmbedding and MHSA; verify output shapes
    "
```

Commit 3. *(Day 2 or 3)* Implement `TransformerBlock` and `VisionTransformer`. Run a forward pass to confirm no errors:

```
git add vit_template.py
git commit -m "implement TransformerBlock and VisionTransformer"
```

Commit 4. *(Day 3)* Implement `get_cifar10_subset` and `train_model`. Run `--mode all` to generate all outputs:

```
git add vit_template.py training_log.json checkpoints/ ablation_logs/
    analysis/
git commit -m "complete Part 1; baseline achieves XX% val acc"
```

Commit 5. *(Day 4 or 5)* Draft and finalize the written report:

```
git add report.pdf
git commit -m "add written report"
```

Note

These are suggestions. Your actual commit messages should describe what you *actually* did and what you observed. Good commit messages are a professional habit: they make it easy to understand your progress and debug problems.

7.3 Useful Git Commands

```
# Check the current status of tracked/untracked files
git status

# See what changes you have made since the last commit
git diff

# Stage a specific file for the next commit
git add vit_template.py

# Stage all changed files in the current directory
git add .

# Commit staged changes with a message
git commit -m "your message here"

# View the commit history (one line per commit)
git log --oneline

# View the commit history with dates
git log --oneline --format="%h %ad %s" --date=short

# Undo changes to a file since the last commit (careful!)
git checkout -- vit_template.py
```

7.4 Including `.git` in Your Zip

By default, many GUI archive tools skip hidden directories (those starting with a dot, like `.git`). You **must** include it. On Unix/macOS use:

```
cd ..   # go one level above your project folder
zip -r UBNumber.zip UBNumber/
```

On Windows (PowerShell):

```
Compress-Archive -Path UBNumber -DestinationPath UBNumber.zip
```

After zipping, verify the `.git` folder is present:

```
unzip -l UBNumber.zip | grep ".git/"
```

7.5 Verifying Your Git History

Run the provided script before submitting:

```
python check_git.py UBNumber/
```

Expected output (example):

```
Commits found: 6
Unique calendar days: 3
PASS: git history requirement satisfied.
```

8 Submission Format

Submit a single zip file named `UBNumber.zip` to the course portal. When unzipped it must produce exactly this directory layout:

```
UBNumber/
    .git/                           <- must be present; see Section 7
    vit_template.py
    training_log.json
    report.pdf                      <- written report (max 6 pages)
    checkpoints/
        baseline_epoch_5.pt
        baseline_epoch_10.pt
        baseline_epoch_20.pt
    ablation_logs/
        patch_2.json
        patch_4.json
        patch_8.json
        layers_2.json
        layers_4.json
        layers_8.json
        no_pos_enc.json
    analysis/
        attention_entropy.json
        pos_embed_correlation.json
```

```
        per_class_accuracy.json
        attention_distance.json
```

Important

The autograder checks for `vit_template.py`, `training_log.json`, and `report.pdf`. Missing files result in lost points. Double-check with:

```
find UBNumber/ -type f | sort
```

and compare against the layout above before submitting.

9 Grading Summary

Part	Component	Points
1	PatchEmbedding	5
1	MultiHeadSelfAttention	10
1	TransformerBlock	5
1	VisionTransformer	5
1	get_cifar10_subset	3
1	train_model	7
2A	Report: patch size ablation	10
2A	Report: depth ablation	8
2A	Report: positional enc. ablation	7
2B	Report: attention entropy interpretation	4
2B	Report: pos. embed. correlation interpretation	4
2B	Report: per-class accuracy interpretation	3
2B	Report: attention distance interpretation	4
3	compute_attention_entropy	6
3	compute_pos_embed_correlation	6
3	compute_per_class_accuracy	5
3	compute_attention_distance	8
Deduction: Git history violation		−15
Deduction: Modified function signatures		−10
Total		**100**

10 Academic Integrity

You may reference: the original ViT paper (Dosovitskiy et al., 2021), the PyTorch documentation, and course lecture notes. You may **not** copy code from existing ViT implementations (timm, HuggingFace Transformers, GitHub repositories, etc.).

Use of large language models (LLMs) to generate implementation code is **not permitted**. You are expected to understand and be able to explain every line you submit.

11 Tips and Common Pitfalls

- **Set `STUDENT_ID` first.** Every numerical result depends on it. Do this before writing any other code.
- **Check tensor shapes frequently.** Insert `print(x.shape)` statements at each stage. The docstrings specify the expected shapes. A shape mismatch in PatchEmbedding will silently corrupt everything downstream.
- **Preserve `.git` when zipping.** See Section 7 for the correct `zip` command.
- **Validate JSON before submitting.** `python -m json.tool file.json` will error immediately if the file is malformed.
- **Report quality.** Every claim in your report should be backed by a specific number from a JSON file (e.g., "layer 3 entropy dropped to 2.44, 24% below layer 0"). Vague observations without quantitative support will not receive full marks.
- **Start early.** The autograder re-runs your training for one epoch. If your code raises an exception, you receive zero points for that component. Debugging at the last minute leaves no time to write a thoughtful report.

References

[1] A. Vaswani et al., "Attention Is All You Need," *NeurIPS*, 2017.

[2] A. Dosovitskiy et al., "An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale," *ICLR*, 2021.

[3] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *NAACL*, 2019.